

Gestione della Memoria

Gestione della Memoria

Introduzione all'allocazione dinamica della memoria con new e delete in C++.

Come funziona la memoria in C++?

Immagina di organizzare una festa. Non sai quante persone verranno. Se prenoti 10 sedie prima, e vengono in 50, hai un problema. Se ne prenoti 200, e vengono in 10, hai sprecato spazio.

La **memoria dinamica** risolve questo problema: puoi "prenotare" esattamente la quantità di memoria che ti serve, nel momento in cui sai quanta ne hai bisogno.

Due tipi di memoria

In C++, la memoria funziona in due modi diversi:

Memoria automatica (stack): le variabili normali. Il computer le crea quando entri in una funzione e le distrugge da solo quando esci. La quantità deve essere nota in anticipo.

```
int x = 5;           // creato automaticamente, distrutto automaticamente
int arr[10];         // array di dimensione fissa – deve essere nota prima
```

Memoria dinamica (heap): tu decidi quanta memoria serve, nel momento in cui ti serve. Devi gestirla tu — compresa la "restituzione" quando hai finito.

```
int* ptr = new int;    // crei uno spazio nella memoria heap
int* arr = new int[100]; // un array di 100 interi – deciso a runtime
```

Pensa all'heap come a un magazzino enorme. Puoi affittare uno spazio, usarlo, e poi restituirlo. Se non lo restituisci, rimane occupato inutilmente.

Allocare memoria: **new**

L'operatore **new** affitta uno spazio nell'heap e ti restituisce l'indirizzo (sotto forma di puntatore):

```
// Alloca spazio per un singolo intero
int* ptr = new int;
*ptr = 42;
cout << *ptr << endl;    // 42

// Alloca e inizializza subito
int* ptr2 = new int(100);
cout << *ptr2 << endl;    // 100

// Alloca un array di 5 interi
int* arr = new int[5];
arr[0] = 10;
arr[1] = 20;
// ...
```

Liberare memoria: **delete**

Ogni volta che usi **new**, devi usare **delete** quando hai finito. Altrimenti la memoria rimane "affittata" per sempre — si chiama **memory leak** (perdita di memoria):

```
int* ptr = new int(42);
cout << *ptr << endl;    // 42

delete ptr;               // "restituisce" la memoria affittata
ptr = nullptr;            // buona abitudine: azzera il puntatore dopo delete
```

Per un array allocato con **new[]**, usa **delete[]** (con le parentesi quadre):

```
int* arr = new int[10];
// ... usa arr ...
delete[] arr;             // IMPORTANTE: delete[] e non solo delete
arr = nullptr;
```

Il memory leak: la perdita di memoria

Se dimentichi il `delete`, la memoria resta occupata finché il programma non chiude:

```
// Ogni volta che chiami questa funzione, perdi 4 byte di memoria
void funzione() {
    int* ptr = new int(42);
    // ... usa ptr ...
    // manca delete ptr! - il ptr viene distrutto ma la memoria no
}

// Se la chiami mille volte, hai perso 4000 byte senza saperlo
for (int i = 0; i < 1000; i++) {
    funzione();
}
```

In un programma piccolo non si nota. In un programma che gira per ore o giorni, i leak si accumulano fino a esaurire la RAM.

Il dangling pointer: il puntatore "fantasma"

Dopo aver chiamato `delete`, il puntatore esiste ancora ma punta a memoria che non è più tua. Usarlo è un errore grave:

```
int* ptr = new int(42);
delete ptr;
// ptr è ora un "puntatore fantasma": punta a memoria liberata

cout << *ptr;    // ERRORE: comportamento non definito (possibile crash)

// Soluzione: dopo delete, metti sempre nullptr
delete ptr;
ptr = nullptr;

if (ptr != nullptr) {
    cout << *ptr;    // non verrà mai eseguito, ma è sicuro
}
```

Esempio pratico: array di dimensione variabile

Il caso più comune in cui serve la memoria dinamica è quando non sai in anticipo quanti elementi hai bisogno:

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Quanti numeri vuoi inserire? ";
    cin >> n;

    // Alloca esattamente n interi – deciso a runtime!
    int* numeri = new int[n];

    // Inserisci i numeri
    for (int i = 0; i < n; i++) {
        cout << "Numero " << (i + 1) << ": ";
        cin >> numeri[i];
    }

    // Calcola la media
    int somma = 0;
    for (int i = 0; i < n; i++) {
        somma += numeri[i];
    }
    cout << "Media: " << (double)somma / n << endl;

    // Libera la memoria!
    delete[] numeri;
    numeri = nullptr;

    return 0;
}
```

La soluzione moderna: puntatori smart

Gestire `new` e `delete` manualmente è complicato e soggetto a errori. In C++ moderno si usano i **puntatori smart**, che si occupano di liberare la memoria automaticamente quando non serve più:

```
#include <memory>
using namespace std;

// unique_ptr: possiede la memoria e la libera automaticamente
unique_ptr<int> ptr = make_unique<int>(42);
cout << *ptr << endl;    // 42
// Non serve delete: viene chiamato automaticamente quando ptr esce dal blocco

// shared_ptr: più puntatori condividono la stessa memoria
shared_ptr<int> sptr = make_shared<int>(100);
```

I puntatori smart eliminano quasi tutti i problemi di memory leak e dangling pointer.

La soluzione ancora più semplice: **vector**

Per gli array dinamici, la scelta migliore è usare **vector** dalla libreria standard. Gestisce la memoria per te:

```
#include <vector>
using namespace std;

// Invece di:
int* arr = new int[n];
// ... usa arr ...
delete[] arr;

// Usa semplicemente:
vector<int> arr(n);
// ... usa arr ...
// nessun delete! viene liberato automaticamente
```

Regole pratiche

- Se usi **new**, usa sempre **delete** nella stessa funzione (o usa uno smart pointer)
- Se usi **new[]**, usa sempre **delete[]** (non **delete**)
- Dopo ogni **delete**, imposta il puntatore a **nullptr**
- Quando puoi, usa **vector** invece di array dinamici manuali

- Per oggetti singoli, usa `make_unique` invece di `new`